

BERT: Frozen Features vs. Full Fine-Tuning on AG News

Sabas Rojas & Daniel Duru
CS 5365 Deep Learning • University of Texas at El Paso

1. Introduction

A central question in NLP transfer learning is how much task-specific adaptation a pretrained model actually needs. This project builds on the work from **Devlin et al. (NAACL 2019)** in *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. This is done by contrasting two fine-tuning regimes on a 4-class text classification task (AG News).

BERT introduced deep *bidirectional* pretraining via masked language modeling (MLM) and next-sentence prediction (NSP) on large unlabeled corpora, producing contextual token representations that transfer broadly across tasks. The paper shows that a single pretrained model, fine-tuned end-to-end, achieves state-of-the-art results across eleven NLP benchmarks (GLUE Table 1). The comparison we aim to measure is whether those representations are rich enough that even a frozen encoder suffices for downstream classification.

2. Chosen Result

The specific result we aim to reproduce is whether full end-to-end fine-tuning outperforms using frozen BERT representations as a fixed feature extractor. This comparison can be seen in **Table 1 (GLUE)** and the ablation in **Section 5.1**, where “BERTLARGE FT” (fine-tuned) consistently dominates “BERTLARGE FE” (features only). We chose this result because it is most practical and important takeaway for practitioners; it directly answers whether encoder weights need updating or whether BERT can be treated as a black-box embedding model. This matters because fine-tuning costs significantly more compute and memory, so if frozen features were good enough, most people would just use that instead.

3. Methodology

Dataset. AG News (Zhang et al., 2015) — 120,000 train / 7,600 test news articles across 4 classes: World, Sports, Business, Sci/Tech. Loaded directly from HuggingFace Datasets with no additional preprocessing beyond tokenization.

Model. bert-base-uncased (12 layers, 768 hidden, 110M parameters) loaded via HuggingFace Transformers, with a randomly-initialized linear classification head (768 → 4).

Two training conditions — identical hyperparameters, one toggle:

- **Frozen (linear probe):** All bert.* parameters frozen (requires_grad = False). Only the 3,076-parameter classification head trains. Reflects “BERT as feature extractor.”
- **Full fine-tuning:** All 109,485,316 parameters update. Reflects the paper’s primary recommended regime.

Shared hyperparameters: AdamW optimizer, lr = 2e-5, batch size = 16, 3 epochs, max token length = 128, seed = 42. Loss logged every 20 steps; test-set accuracy and sklearn weighted F1 computed after each epoch.

Differences from the paper. Devlin et al. evaluate on GLUE tasks (SST-2, MRPC, etc.) using BERT LARGE. We use bert-base-uncased on AG News with a single seed and no hyperparameter search. These are accepted resource-driven simplifications consistent with the project scope.

4. Results & Analysis

Mode	Test Accuracy	Weighted F1	Trainable Params
Frozen (linear probe)	74.04%	73.87%	3,076
Full fine-tuning	94.63%	94.64%	109,485,316

Table 1. Test-set results after 3 epochs. Full fine-tuning yields a +20.6 pp accuracy gain over the frozen linear probe.

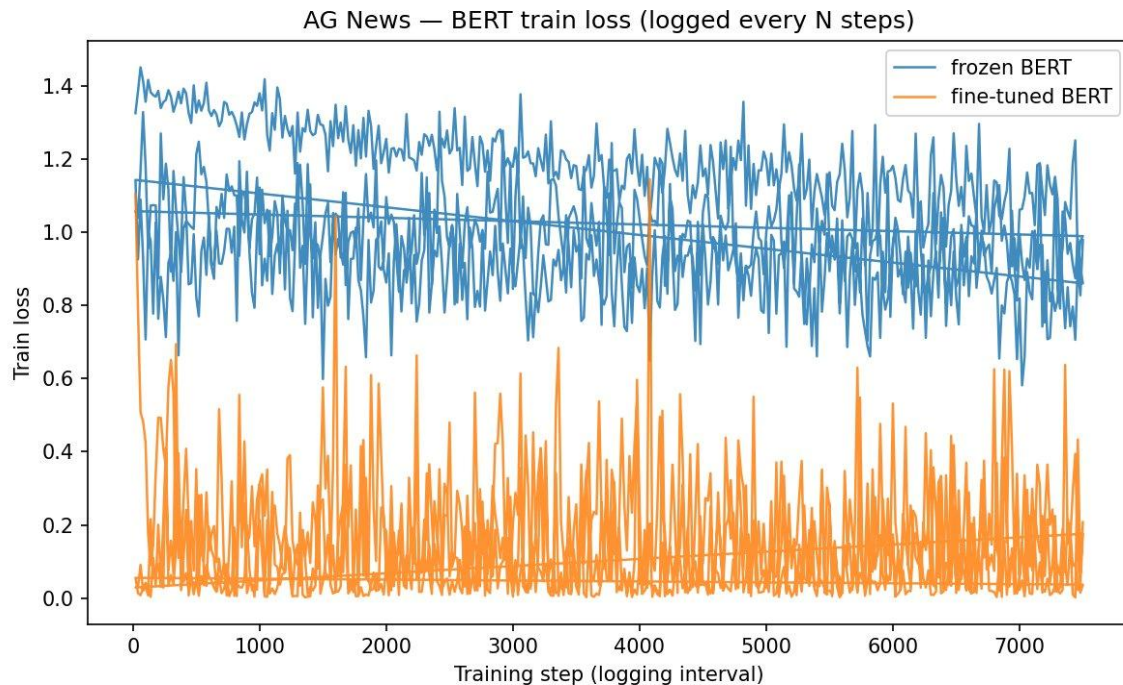


Figure 1. Per-step training loss across all 3 epochs. Blue: frozen BERT (slow, noisy descent, plateau ~ 0.88). Orange: full fine-tuning (rapid convergence in epoch 1, stabilizes near 0.09 by epoch 3).

Discussion. The ~ 21 percentage-point accuracy gap is large and consistent with the paper’s findings. Figure 1 makes this concrete: fine-tuning drops from ~ 1.1 to near-zero loss within the first few hundred steps of epoch 1, while the frozen model descends slowly and noisily across all three epochs, never approaching those loss levels. This reflects the frozen model being limited to finding a hyperplane in a fixed 768-dimensional space, whereas fine-tuning reshapes those representations toward the task.

Discrepancies. A direct numerical comparison to GLUE Table 1 is not possible since our dataset, scale (base vs. large), and task differ. Within our controlled setup, the qualitative conclusion — fine-tuning beats frozen features by a wide margin — replicates cleanly. One unexpected observation: fine-tuning accuracy peaks sharply at epoch 1 (94.04%) and barely changes in epochs 2–3 (94.46%, 94.43%), suggesting AG News is close to saturated at this scale and learning rate, and that additional epochs add cost without meaningful gain.

5. Reflections

- **Colab runtime management is real overhead.** GPU sessions reset without warning; we lost one run mid-way. Future work: checkpoint model weights after each epoch and save to Google Drive so a reset is not catastrophic.
- **The frozen ceiling was lower than expected.** We anticipated ~ 85 – 88% for the linear probe; the actual 74% suggests the head did not fully converge in 3 epochs at $\text{lr} = 2\text{e-}5$. A frozen-only run with a higher head learning rate or more epochs would likely close some of the gap.
- **Single-seed results are fragile.** One seed-42 run cannot rule out lucky or unlucky initialization. Three seeds with averaged results would meaningfully strengthen the claim.
- **Potential extensions:** (1) Partial fine-tuning — unfreeze only the top N layers — to trace the accuracy vs. compute tradeoff continuously. (2) Replicate on SST-2 or MRPC to stay closer to the paper’s actual benchmarks. (3) Add a per-class confusion matrix to identify which AG News categories the frozen model struggles with most.

6. References

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL 2019*.
- Wolf, T. et al. (2020). HuggingFace Transformers: State-of-the-Art Natural Language Processing. *EMNLP 2020 (Systems Demo)*.
- Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *NeurIPS 2015*. (AG News dataset.)
- Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830. (Weighted F1 metric.)